Experiment No.:5

Name of Experiment: Use R as a calculator.

Aim: To make a calculator in R.

Source Code: Basic Calculations in R

1. Basic Arithmetic Operations

```
#Addition
add_result <- 15 + 7

#Subtraction
sub_result <- 20 - 8

#Multiplication
mul_result <- 6 * 9

#Division
div_result <- 81 / 9

#Exponentiation (raising to the power)
exp_result <- 5^3   #5 raised to the power of 3

#Modulus (remainder of division)
mod_result <- 17 %% 5

#Integer division (quotient without remainder)
int_div_result <- 17 %/% 5

#Square root
sqrt_result <- sqrt(144)

#Logarithm (log base e)
log_result <- log(2.718)

#Trigonometric functions (sin, cos, tan)
sin_result <- sin(pi / 2)   #sin of 90 degrees (in radians)
cos_result <- cos(0)        #cos of 0 degrees (in radians)
tan_result <- tan(pi / 4)   #tan of 45 degrees (in radians)

#Print the results
print(paste("Addition: 15 + 7 =", add_result))
print(paste("Subtraction: 20 - 8 =", sub_result))
print(paste("Multiplication: 6 * 9 =", mul_result))
print(paste("Division: 81 / 9 =", div_result))
print(paste("Exponentiation: 5^3 =", exp_result))
print(paste("Modulus: 17 %% 5 =", mod_result))
print(paste("Integer Division: 17 %/% 5 =", int_div_result))
print(paste("Square Root: sqrt(144) =", sqrt_result))
print(paste("Logarithm of e: log(2.718) =", log_result))
print(paste("Sine of 90 degrees: sin(pi/2) =", sin_result))
print(paste("Cosine of 0 degrees: cos(0) =", cos_result))
print(paste("Tangent of 45 degrees: tan(pi/4) =", tan_result))
```

Output:

```
 [1] "Addition: 15 + 7 = 22"
[1] "Subtraction: 20 - 8 = 12"
[1] "Multiplication: 6 * 9 = 54"
[1] "Division: 81 / 9 = 9"
[1] "Exponentiation: 5^3 = 125"
[1] "Modulus: 17 %% 5 = 2"
[1] "Integer Division: 17 %/% 5 = 3"
[1] "Square Root: sqrt(144) = 12"
[1] "Logarithm of e: log(2.718) = 1.000422"
[1] "Sine of 90 degrees: sin(pi/2) = 1"
[1] "Cosine of 0 degrees: cos(0) = 1"
[1] "Tangent of 45 degrees: tan(pi/4) = 1"
```

Basic Level Questions

1. Which symbol is used for exponentiation in R?
   Answer: ^
2. Write the R command to add 10 and 5.
   Answer: 10 + 5
3. What does **%%** operator do in R?
   Answer: Returns the remainder of division (modulus)
4. Which function is used to compute square root in R?
   Answer: sqrt()
5. What is the output of **cos(0)** in R?
   Answer: 1

Moderate Level Questions

1. Write R code to compute 25 divided by 4 and get only the integer result.
   Answer: 25 %/% 4
2. What is the output of **log(exp(1))** in R?
   Answer: 1
3. Write R code to find the sine of 90 degrees in radians.
   Answer: sin(pi/2)
4. What is the output of **17 %% 5**?
   Answer: 2
5. Write R code to calculate 7 multiplied by 8.
   Answer: 7 * 8

Advanced Level Questions

1. **Write a script in R that takes two numbers and returns their sum, product, and quotient.**
   Answer:

   ```
   a <- 10
   b <- 5
   cat("Sum:", a + b, "\n")
   cat("Product:", a * b, "\n")
   cat("Quotient:", a / b)
   ```
2. Evaluate the output of the following: **tan(pi/4) * sqrt(16).**
   Answer: 1 * 4 = 4
3. Create a calculator script in R that performs all basic arithmetic operations on two numbers.
   Answer: Use variables x and y, then perform +, -, *, /, %%, %/%, and ^.
4. Which trigonometric function would return 0 when the input is pi?
   Answer: sin(pi)
5. Write R code to compute the cube of 4 using exponentiation.
   Answer:

Use Cases:

1. Performing Basic Arithmetic Operations
2. Using Trigonometric Functions for Calculations
3. Evaluating Logarithmic and Exponential Expressions
4. Creating a Script-Based Arithmetic Calculator
5. Computing Integer Division and Remainders

Experiment No.:6

Name of Experiment: Explore various functionalities of Data frames.

Aim: To Create and manipulate the Data frames in R.

A **data frame** is a powerful and commonly used data structure in R that stores data in a tabular format, similar to an Excel spreadsheet. Each column can have a different data type (numeric, character, factor, etc.). Data frames are useful for data manipulation, analysis, and visualization. In this use case, we'll explore the various functionalities of data frames, such as creating, accessing, filtering, and manipulating data.

Source Code:

Exploring Data Frames

1. Creating a Data Frame

You can create a data frame manually using data.frame().

```
 Creating a data frame
students <- data.frame(
  Name = c("Alice", "Bob", "Charlie", "David", "Eva"),
  Age = c(24, 22, 23, 25, 21),
  Grade = c("A", "B", "A", "C", "B"),
  Passed = c(TRUE, TRUE, TRUE, FALSE, TRUE)
)

 View the data frame
print(students)
```

Output:

```
    Name Age Grade Passed
1  Alice  24     A   TRUE
2    Bob  22     B   TRUE
3 Charlie  23     A   TRUE
4  David  25     C  FALSE
5    Eva  21     B   TRUE
```

2. Accessing Data in a Data Frame

You can access specific columns, rows, or individual values from the data frame.

```
#Accessing a specific column
print(students$Name)

#Accessing a specific row (3rd row)
print(students[3, ])
```

```
#Accessing a specific value (2nd row, 1st column)

print(students[2, 1])

#Using column name and row number to access data
print(students[4, "Grade"])
```

Output:

```
 [1] "Alice"   "Bob"     "Charlie" "David"   "Eva"

     Name Age Grade Passed
3 Charlie  23     A   TRUE

[1] "Bob"

[1] "C"
```

3. Adding a New Column

You can add new columns to a data frame.

```
 Adding a new column (Gender)
students$Gender <- c("F", "M", "M", "M", "F")

 View the updated data frame
print(students)
```

Output:

```
     Name Age Grade Passed Gender
1   Alice  24     A   TRUE      F
2     Bob  22     B   TRUE      M
3 Charlie  23     A   TRUE      M
4   David  25     C  FALSE      M
5     Eva  21     B   TRUE      F
```

4. Filtering Rows Based on Conditions

You can filter rows based on certain conditions.

```
 Filtering students who passed
passed_students <- students[students$Passed == TRUE, ]

 View the filtered data frame
print(passed_students)
```

Output:

```
     Name    Age Grade Passed Gender
1   Alice     24     A    TRUE      F
2     Bob     22     B    TRUE      M
3 Charlie  23     A    TRUE      M
5     Eva     21     B    TRUE      F
```

5. Summary and Structure of the Data Frame

You can summarize the data frame to get descriptive statistics and explore the structure.

```
Summary statistics of the data frame
summary(students)

Structure of the data frame
str(students)
```

Output:

```
     Name        Age          Grade     Passed  Gender
 Alice   :1   Min.   :21.0   A:2     Mode :logical   F:2
 Bob     :1   1st Qu.:22.0   B:2     FALSE:1         M:3
 Charlie :1   Median :23.0   C:1     TRUE :4
 David   :1   Mean   :23.0
 Eva     :1   3rd Qu.:24.0
              Max.   :25.0
'data.frame':     5 obs. of  5 variables:
 $ Name  : chr  "Alice" "Bob" "Charlie" "David" ...
 $ Age   : num  24 22 23 25 21
 $ Grade : chr  "A" "B" "A" "C" ...
 $ Passed: logi  TRUE TRUE TRUE FALSE TRUE
 $ Gender: chr  "F" "M" "M" "M" ...
```

6. Modifying Existing Data

You can modify specific values in the data frame.

```
Modify David's grade from C to B
students[students$Name == "David", "Grade"] <- "B"

View the updated data frame
print(students)
```

Output:

```
     Name Age Grade Passed Gender
1   Alice  24     A   TRUE      F
2     Bob  22     B   TRUE      M
3 Charlie  23     A   TRUE      M
4   David  25     B  FALSE      M
5     Eva  21     B   TRUE      F
```

7. Deleting Rows or Columns

You can remove specific rows or columns from a data frame.

```
Remove the "Passed" column
students <- students[, -which(names(students) == "Passed")]

Remove the 4th row (David)
students <- students[-4, ]

View the updated data frame
print(students)
```

Output:

```
     Name Age Grade Gender
1   Alice  24     A      F
2     Bob  22     B      M
3 Charlie  23     A      M
5     Eva  21     B      F
```

8. Sorting a Data Frame

You can sort the data frame by a specific column (e.g., Age).

```
 Sort students by Age in ascending order
sorted_students <- students[order(students$Age), ]

 View the sorted data frame
print(sorted_students)
```

Output:

```
     Name    Age Grade Gender
5     Eva     21     B      F
2     Bob     22     B      M
3 Charlie 23     A      M
1   Alice     24     A      F
```

Basic Level Questions

1. Which function is used to create a data frame in R?
   Answer: data.frame()
2. How do you access the 2nd row and 1st column in a data frame?
   Answer: df[2, 1]
3. Which symbol is used to access a column in a data frame?
   Answer: $
4. What function gives a summary of a data frame?
   Answer: summary()
5. How do you add a new column to a data frame?
   Answer: df$new_column <- values

Moderate Level Questions

1. Write R code to filter students who passed the exam.
   Answer: `df[df$Passed == TRUE, ]`
2. Write R code to change the value of a specific cell in a data frame.
   Answer: `df[df$Name == "David", "Grade"] <- "B"`
3. How do you remove the 4th row from a data frame?
   Answer: `df <- df[-4, ]`
4. What function helps understand the structure of a data frame?
   Answer: `str()`
5. How do you access only the 'Grade' column from the data frame?
   Answer: `df$Grade`

Advanced Level Questions

1. Write R code to sort a data frame by Age in descending order.
   Answer:

```
sorted_df <- df[order(-df$Age), ]
```

2. Create a new column named 'Score' and assign random values.
   Answer:

```
df$Score <- c(85, 90, 78, 88, 92)
```

3. Write R code to remove a column named "Passed".
   Answer:

```
df <- df[, -which(names(df) == "Passed")]
```

4. Combine filtering and sorting: show students who passed and sort them by Age.
   Answer:

```
passed_sorted <- df[df$Passed == TRUE, ]
passed_sorted <- passed_sorted[order(passed_sorted$Age), ]
```

5. How do you update multiple columns at once in a data frame?
   Answer: Use indexing with column names or position, e.g.:

```
df[1, c("Grade", "Gender")] <- c("A", "F")
```

Use Cases:

1. Creating Data Frames
2. Accessing Rows, Columns, and Elements
3. Adding New Columns
4. Filtering Data Based on Conditions
5. Summarizing and Understanding Structure
6. Modifying Existing Data
7. Deleting Rows or Columns
8. Sorting Data Frames

Experiment No.:7

Name of Experiment: Create data set using data frames, list and tables

Aim:

Source Code:

1. Creating a Data Set Using Data Frames

A **data frame** is a two-dimensional table-like data structure where columns can contain different types of data (numeric, character, logical, etc.). Here's an example where we create a dataset for students' information, including names, ages, grades, and whether they passed.

Source Code:

```
 Creating a data frame
students_df <- data.frame(
  Name = c("Alice", "Bob", "Charlie", "David", "Eva"),
  Age = c(24, 22, 23, 25, 21),
  Grade = c("A", "B", "A", "C", "B"),
  Passed = c(TRUE, TRUE, TRUE, FALSE, TRUE)
)

 View the data frame
print(students_df)
```

Output:

```
    Name Age Grade Passed
1   Alice  24     A   TRUE
2     Bob  22     B   TRUE
3 Charlie  23     A   TRUE
4   David  25     C  FALSE
5     Eva  21     B   TRUE
```

2. Creating a List

A **list** is a versatile data structure in R that can contain elements of different types, including other lists, vectors, or data frames. Let's create a list that stores multiple types of information about a class, such as students' data, the class name, and the total number of students.

Source Code:

```
 Creating a list
class_info <- list(
  ClassName = "Data Science 101",
  Students = students_df,    Storing the data frame inside the list
  TotalStudents = nrow(students_df),
  Instructor = "Dr. Smith"
)

 View the list
print(class_info)
```

Output:

```
$ClassName
[1] "Data Science 101"

$Students
      Name Age Grade Passed
1    Alice  24     A   TRUE
2      Bob  22     B   TRUE
3  Charlie  23     A   TRUE
4    David  25     C  FALSE
5      Eva  21     B   TRUE

$TotalStudents
[1] 5

$Instructor
[1] "Dr. Smith"
```

3. Creating a Table (Contingency Table)

A **table** is a data structure that is particularly useful for summarizing categorical data and counting occurrences. Let's create a table to summarize the number of students in each grade category.

Source Code:

```
Creating a table (contingency table) to summarize grades
grades_table <- table(students_df$Grade)


View the table
print(grades_table)
```

Output:

```
A B C
2 2 1
```

This shows there are 2 students with an "A" grade, 2 with a "B", and 1 with a "C".

4. Using the Data Structures in an Experiment

Experiment: Analyzing and Summarizing Student Performance

Let's say we're running an experiment where we need to analyze the performance of students based on their grades and whether they passed. We'll use the data frame to filter and manipulate data, the list to store multiple types of information about the class, and the table to summarize the grades.

Source Code:

```
Step 1: Filter students who passed
passed_students <- students_df[students_df$Passed == TRUE, ]
print("Students who passed:")
print(passed_students)


Step 2: Calculate the average age of students
average_age <- mean(students_df$Age)
print(paste("Average age of students:", average_age))


Step 3: Update the list with the average age
class_info$AverageAge <- average_age
print("Updated class info with average age:")
```

```
print(class_info)

 Step 4: Summary of grades using the table
print("Summary of grades:")

print(grades_table)
```

Output:

```
 [1] "Students who passed:"
       Name Age Grade Passed
1     Alice  24     A   TRUE
2       Bob  22     B   TRUE
3   Charlie  23     A   TRUE
5       Eva  21     B   TRUE

[1] "Average age of students: 23"

[1] "Updated class info with average age:"
$ClassName
[1] "Data Science 101"

$Students
     Name  Age Grade Passed
1   Alice   24     A   TRUE
2     Bob   22     B   TRUE
3 Charlie   23     A   TRUE
4   David   25     C  FALSE
5     Eva   21     B   TRUE

$TotalStudents
[1] 5
```

```
$Instructor
[1] "Dr. Smith"

$AverageAge
[1] 23

[1] "Summary of grades:"

A B C
2 2 1
```

Explanation of the Experiment:

1. **Filter the Data**:
   - We filtered the students who passed the course using the students_df data frame.
2. **Calculate Summary Statistics**:
   - We calculated the average age of students using the mean() function.
3. **Update the List**:
   - We added the average age to the class_info list, demonstrating how lists can store different types of data and how they can be updated dynamically.
4. **Summarize Grades**:
   - Using the table() function, we summarized how many students achieved each grade, providing a quick overview of the distribution of grades.

Basic Level Questions

1. Which function is used to create a data frame in R?
   Answer: data.frame()
2. How do you access an element from a list in R?
   Answer: list_name$element_name or list_name[["element_name"]]
3. What function is used to create a table of frequency counts in R?
   Answer: table()
4. Which function calculates the number of rows in a data frame?
   Answer: nrow()
5. How can you view the structure of a data frame?
   Answer: str()

Moderate Level Questions

1. Write R code to filter students who passed.
Answer:

```
students_df[students_df$Passed == TRUE, ]
```

2. How do you calculate the average age of students?
Answer:

```
mean(students_df$Age)
```

3. How can you add a new field to an existing list in R?
Answer:

```
list_name$new_field <- value
```

4. Write code to count how many students got grade "A".
Answer:

```
table(students_df$Grade)["A"]
```

5. How do you update a specific value in a data frame (e.g., David's grade)?
Answer:

```
students_df[students_df$Name == "David", "Grade"] <- "B"
```

Advanced Level Questions

1. Write code to append the average age to an existing list.
Answer:

```
class_info$AverageAge <- mean(students_df$Age)
```

2. Write R code to filter and display only students with grade “B” who passed.
Answer:

```
students_df[students_df$Grade == "B" & students_df$Passed == TRUE, ]
```

3. How do you create a nested list that includes a data frame and a summary table?
Answer:

```
new_list <- list(Students = students_df, GradeSummary = table(students_df$Grade))
```

4. Write R code to calculate and store summary statistics for each column in the data frame.
Answer:

```
summary(students_df)
```

5. How do you combine the student data frame with a new student’s information?
Answer:

```
new_student <- data.frame(Name="Frank", Age=23, Grade="A", Passed=TRUE)
students_df <- rbind(students_df, new_student)
```

Use Cases:

1. Creating a Student Dataset Using Data Frame
2. Storing Metadata with List Objects
3. Summarizing Categorical Data Using Tables
4. Dynamic Update of Information in List
5. Filtering and Statistical Analysis on Data Frames

Experiment No.:8

Name of Experiment: Create the contingency table for the given raw data

Aim: creates a contingency table from raw data is to analyze and summarize the relationship between two categorical variables.

To create a contingency table from raw data in R, we typically need two categorical variables. For this example, let's assume we have data regarding gender and the choice of a preferred study method among students. Here's how to create a contingency table, along with the source code, output, and potential use cases.

Example Raw Data

Let's say we have the following data on students:

Male	Online
Female	In-person
Male	In-person
Female	Online
Male	Online
Female	Online
Male	In-person
Female	In-person

SOURCE CODE:

```
 Step 1: Create the raw data
data <- data.frame(
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female"),
  Study_Method = c("Online", "In-person", "In-person", "Online", "Online", "Online", "In-
person", "In-person")
)

 Step 2: Create the contingency table
contingency_table <- table(data$Gender, data$Study_Method)

 Step 3: Display the contingency table
print(contingency_table)
```

OUTPUT:

	In-person	Online
Female	2	3
Male	2	3